

המהדר קורא את הקוד שלך

שבוע: 12 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 13

משקל בציר הסופי: מטלת כיתה 12 · חלק א · חמישים אחוז מציון המטלה · 10 הטובות מתוך 12 נסמנות **משוער**: 10 דקות בכיתה, והשלמה בבית

מטרה

להעביר שכבה אחת — שכבת הרשת — לטיפוסים, **בלי לשנות שום דבר שהיישום עושה**, ולצאת עם משפט אחד שאפשר לבדוק: הרצת את המהדר, והוא מצא הנחה שגויה על התשובה של מישהו אחר לפני שהרצת שורה.

היום הקלדת את ההעברה הזאת פעם אחת, על ספריית ההשאלה: `storage.js` עבר ל-`ts/storage.ts` עם `unknown` בגבול ושומר `value is ShelfEntry`, העוזר `$` עם `<T extends Element = Element>` עשה את ההצהרה פעם אחת, ו-Claude העביר את `api.js` — ומחקת שלוש השתקות שהמהדר קיבל בשקט. **המטלה היא אותה העברה, על `js/api.js` של המזוהה**, בשמות אחרים: `ShelfEntry` הוא `Item`, `LookupError` הוא `ApiError`, `isEntry` הוא `isItem`. שום דבר לא מועתק; הכול מזוהה.

זו לא כתיבה מחדש. **ההעברה הזאת היא שינוי משמר-התנהגות** — המונח ממחזור 4 — ויש לה בדיקה אחת שסוגרת אותה: `diff` בין הפלט שנוצר לבין הקובץ שקיבלת. מה שאמור להיות שונה שם הוא סוגריים, רווחים — ובדיקת `typeof` שהמהדר צריך כדי לצמצם `unknown` (למשל `typeof page.index === 'number'` לפני `Number.isFinite(page.index)`), שמוסיפה לתנאי ביטחון ולא משנה את התוצאה שלו. **תנאי שהתוצאה שלו השתנתה — לא.**

וזו גם המטלה שמסמנת V על דרישות 22–24 של הפרויקט, שמוגש בסוף שיעור 12.

הזמן: עשר דקות בכיתה (165–175) — **חלק א, המפרט, לפני כל כלי AI**, וקומיט שלו. הניתוח המלא לוקח חמישים דקות מהבלוק, ולכן כל השאר בבית, עד תחילת שיעור 13.

מה כבר הקלדת היום

החלק	איפה הקלדת אותו	מה משתנה במזוהה
א · המפרט	מחזור 4 — שלוש שורות על הלוח לפני המהלך הראשון	חמישה סעיפים, בקומיט שלפני <code>ts/</code>
ב · הטיפוסים והשומר	מחזור 1 — <code>ShelfEntry</code> , <code>LookupKind</code> , <code>isRecord</code> , <code>isEntry</code> ; <code>you-do</code> שלו — <code>isBook</code>	<code>Item</code> , <code>Suggestion</code> , <code>ErrorKind</code> , <code>isItem</code> , <code>RequestStatus</code>
ג · ההעברה	מחזור 1 — <code>unknown</code> בגבול, <code>'./types.js'</code> ; מחזור 3 — <code>readonly kind</code> , <code>res.json()</code> ל- <code>unknown</code>	<code>ts/api.ts</code> , מאותו <code>api.js</code>
ד · ה-DOM	מחזור 2 — <code>\$</code> עם <code><T extends Element = Element></code> , <code>inputText</code> , הייבוא ב- <code>app.js</code>	<code>ts/dom.ts</code> , וקובץ JavaScript אחד שמייבא ממנו
ה · הביקורת	מחזור 3 — שורת סקירה, בצורה של הדוגמה	שני דברים ב- <code>PROMPTS.md</code>

תוצרי למידה

1. לדעת מה טיפוסים תופסים ומה לא — ולומר את שני הצדדים.

2. להבחין בין `unknown` ל-`any`, ולדעת למה `as` אינו בדיקה.
3. לכתוב שומר בזמן ריצה שמצמצם (`value is T`), ולא רק מחזיר `boolean`.
4. להקליד שאילתת DOM ומטפל אירוע בלי לפזר `as HTMLInputElement` בארבעים מקומות.
5. להריץ `tsc` עם `tsconfig` מינימלי, ולהבין מה `strict` באמת קונה.
6. לעבור מ-`js` ל-`ts`. **בהדרגה**, מודול אחד, בלי לעצור את שאר היישום.

מה יש בתיקייה

קובץ	מה עושים איתו
<code>js/api.js</code>	הקובץ של היום. הוא עובר ל- <code>ts/</code> , והוא נמחק מ- <code>js/</code> בידיים? לא — קרא את הערה למטה
<code>ts/types.ts</code>	שלד. הישות, האיחודים, והשומר
<code>ts/dom.ts</code>	שלד. שני עוזרים גנריים. חלק ד
<code>tsconfig.json</code>	נתון ומלא. לא נוגעים בו — אותו קובץ כמו במחזורים
<code>specs/ts-migration.md</code>	הקובץ הראשון שאתה פותח היום. חלק א
<code>js/</code> שאר	נשאר JavaScript. לא נוגעים בו — חוץ משורת ייבוא אחת בחלק ד

`js/api.js` נמחק, ואז נולד מחדש

`tsconfig.json` מגדיר `"rootDir": "ts"` ו-`"outDir": "js"`. ברגע שתיצור `ts/api.ts` ותריץ את המהדר, הוא יכתוב `js/api.js` **בעצמו** — ויחליף את הקובץ הישן. זה מכון: זה אותו קובץ, ולכן אותו נתיב, ולכן `render.js` ו-`events.js` לא צריכים לדעת שקרה משהו.

פתח את הפלט אחרי הבנייה הראשונה והסתכל בו. זו החוויה שכל השיעור מסתמך עליה.

המהדר מותקן **פעם אחת, גלובלית** — `npm install --global typescript@5.9.3`, כמו ב-`pre-class/` — ו-`tsc` `npx` מוצא אותו בלי רשת.

מגבלות – מה אסור השבוע

any לא מופיע בשום קובץ ב- `ts/`. לא כפתרון זמני, לא ב- `catch`, לא באף מקום. `unknown` מותר, רצוי, וממש לא אותו דבר.

מה שיוצא מ- `res.json()` מוקלד `unknown` ולא ב- `as X`. `await res.json()` מהדר, קצר יותר, ובדוק בדיוק כלום. זו השורה שהמטלה הזאת קיימת בשבילה.

strict נשאר **true**. אם כיבית אותו כדי שיעבור – לא העברת, ויתרת.

אין `@ts-ignore` ואין `@ts-expect-error`. שגיאה שהמהדר מצא – מתקנים אותה, לא משתיקים.

טיפוס שמתאר את הסכמה של ה-API שלך מוצהר ב- `ts/api.ts` ואינו מיוצא. אף קובץ אחר ביישום לא אמור להכיר את שמות השדות של מישורו אחר.

אין שינוי התנהגות. לא הודעה, לא תוצאה של תנאי, לא סדר. אם שיפרת משהו בדרך – החזר אותו. (בדיקת `typeof` שהמהדר דורש לפני שדה של `unknown` אינה שינוי: התנאי מחזיר אותו דבר.)

הדפדפן טוען את `js/`, לא את `ts/`. אל תשנה אף `<script src>`.

חלק א · המפרט, שנכתב ראשון · 12 נקודות ליבה

פתח את `specs/ts-migration.md` לפני שאתה פותח כלי AI – בכיתה, בעשר הדקות. חמישה סעיפים, והם קצרים בכוונה – מפרט שלוקח עשרים דקות לכתוב הפסיק להיות מפרט.

הסעיף	מה הוא מכריח אותך להחליט
1 · מה אני מעביר ומה לא	שההעברה היא הדרגתית, ושאתה יודע איפה היא נעצרת
2 · הטיפוסים שהקוד צריך	<code>interface</code> או <code>type</code> לכל אחד, ולמה דווקא זה
3 · הגבול	איפה נכנס ערך שלא כתבת, ומה מצמצם אותו
4 · מה חייב להישאר זהה	שלוש התנהגויות, ואיך תבדוק כל אחת
5 · הבקשה, ומה תבדוק בפלט	ארבעה דברים קונקרטיים. "לבדוק שזה עובד" אינו אחד מהם

למה זה נבדק, ולמה זה שווה 12 נקודות

מפרט שנכתב אחרי הקוד תמיד מסכים איתו ולעולם לא מזהיר מפני כלום. הראיה היא ההיסטוריה: קומיט של המפרט, ואז קומיט של `ts/`. בסעיף 4 אתה כותב מה אסור שישתנה – כמו שלוש השורות על הלוח במחזור 4 – ובסעיף 5 אתה כותב ארבע טעויות שאתה מצפה שהמכונה תעשה, לפני שהיא עשתה אותן. ראית שלוש מהן במחזור 3. אלה שני הדברים היחידים במטלה שאי אפשר לייצר בדיעבד.

חלק ב · הטיפוסים והשומר · 26 נקודות ליבה

`ts/types.ts`. כתוב אותו ראשון – כל השאר מחזיר משהו שמתואר כאן.

ב.1 · הצורות

- **Item** — הפריט שלך. העתק את שמות השדות מ- `js/items.js`; הם כבר קיימים.
- **Suggestion** — הצעה **אחרי נרמול**. לא מה שה-API שולח.
- **ErrorKind** — ארבע הסיבות לכישלון, כאיחוד של ליטרלים.
- **RequestStatus** — ארבעת המצבים, כאיחוד.

interface או **type**? ההחלטה שלך, ואתה מנמק אותה במפרט. הכלל שעובד: `interface` לצורה של אובייקט שקוד אחר צורך; `type` לאיחוד, לחיתוך, ולכינוי. **איחוד לא ניתן לכתוב כ- interface בכלל** — זו לא העדפה, זו יכולת. (הבודק קורא איחוד בכל מירכאות וגם כשהוא שבור על כמה שורות, ושדות בכל הזחה).

ב.2 · השומר

```
export function isItem(value: unknown): value is Item;
```

המילה is היא כל ההבדל. `boolean` אומר "התשובה כן"; `value is Item` אומר "ומכאן והלאה המהדר רשאי להתייחס אליו כ- Item". זה הגשר היחיד מ- `unknown` לטיפוס אמיתי שאינו שקר. `function` או `arrow` — שניהם נקראים כשומר. **והבודק סופר: בגוף השומרים יחד — לפחות שלוש בדיקות** `typeof` **על שדות** (`typeof value.title === 'string'`). שומר שמחזיר `true` מיד עובר את המהדר ואינו שומר על כלום.

קפדן על מה שבלעדיו הציור נשבר, מתירני על מה שמוצג בתנאי. שומר שפוסל פריט שלם בגלל שדה אופציונלי חסר זרק נתונים שהמשתמש היה יכול להשתמש בהם — ובדיוק את זה עשית עם `isBook` ושנה שחסרה.

למה שומר בכלל, אם יש טיפוסים

הטיפוסים נמחקים בקומפילציה. פתח את `js/types.js` אחרי הבנייה הראשונה ותספור מה נשאר: השומרים, ושום דבר אחר. `Item` מתאר מה **אמור** להיות ב-`localStorage`; השומר הוא הקוד היחיד שמגלה מה באמת יש שם. הערך שיצא מהדיסק נכתב על ידי גרסה קודמת של הקוד שלך, או על ידי מישהו עם קונסולה פתוחה.

חלק ג · ההעברה · 34 נקודות ליבה

`js/api.js` עובר להיות `ts/api.ts`. **אותו קובץ, אותם תנאים, אותו סדר.**

מה	איך
חתימות מוקלדות על כל פונקציה מיוצאת	פרמטרים וטיפוס החזרה . אל תיתן להסקה לעשות את זה בשקט
שדות <code>ApiError</code> מוצהרים	<code>readonly kind: ErrorKind</code> — ה-JSDoc היה תגובה; את זה המהדר בודק
<code>res.json()</code> מוקלד <code>unknown</code>	השורה שהכול תלוי בה . לא <code>as</code>
הסכמה של ה-API שלך מוצהרת ומקומית	<code>interface</code> , ב- <code>api.ts</code> , בלי <code>export</code>
<code>catch (error)</code> הוא <code>unknown</code> תחת <code>strict</code>	<code>error instanceof Error</code> לפני <code>error.name</code> . זה לא טקס, זה הבדיקה
ייבוא מ- <code>types.ts</code>	<code>from './types.js'</code> — עם js , גם בקובץ <code>ts</code> : זו הכתובת שהדפדפן יבקש

המלכודת שתפגוש, והיא לא בקוד שלך

`res.json()` מוגדר בטיפוסי ה-DOM כמחזיר `Promise<any>` — `any`, לא `unknown`. כלומר בלי הצהרה מפורשת, `payload.query.pages[0].title.toUpperCase()` עובר את המהדר בלי מילה, על תשובה של 404. `let payload: unknown;` היא שורה אחת, והיא מחזירה את הגבול למקום. (`unknown` : או `as unknown` — שניהם מסמנים את הגבול; `as unknown as X` הוא ביטול כפול, לא גבול).

והמלכודת השנייה, שראית במחזור 3

כשתשנה את שם הקובץ ל-`.ts`, המהדר יגיד `.Property 'kind' does not exist on type 'ApiError'`. התיקון הוא הצהרה — `readonly kind: ErrorKind;` בראש המחלקה — ולא `@ts-ignore`: השתקתה מעבירה את השגיאה לכל מי שיקרא את השדה.

מה שבודקים בסוף, ואתה יכול לבדוק בעצמך

```
npx tsc -p tsconfig.json
git diff --stat js/api.js
```

הראשונה חייבת לעבור, עם `strict` דלוק. השנייה משווה את הפלט שנוצר לגרסה שקיבלת: סוגריים ורווחים — בסדר; בדיקת `typeof` שנוספה כדי שהמהדר יצמצם `unknown` — בסדר, היא לא משנה את תוצאת התנאי; תנאי שהתוצאה שלו השתנתה — לא.

שלוש ההתנהגויות שחייבות להישאר: חיפוש מוצלח מגיע למסך 404 מגיע כשגיאה ולא כתוצאות. הקלדה מהירה לא מאדימה את הפאנל אף פעם.

חלק ד · להקליד את ה-16 DOM נקודות הרחבה

`ts/dom.ts`. הגנריקה היחידה בקורס, והיא שווה את הזמן — הקלדת אותה במחזור 2.

`document.querySelector('#q')` הוא `Element | null`, ושני החצאים בדרך: ל-`Element` אין `value`, ו-`null` | נכון בכל שאלתה — כלומר ארבעים ענפי `null` שאיש לא כותב.

התיקון השגוי הוא `as HTMLInputElement` בכל מקום: הוא מכבה את המהדר בדיוק בגבול שבו הוא היה שווה משהו, בארבעים מקומות שכבר אי אפשר למצוא.

```
export function $<T extends Element = Element>(selector: string, root?: ParentNode): T;
```

`<T extends Element = Element>` נקרא: `T` הוא סוג אלמנט כלשהו, מי שקורא בוחר איזה, וברירת המחדל היא `Element` — וזה מה ששומר על `('main')` עובד בלי ארגומנט טיפוס. ואם האלמנט לא שם — לזרוק, עם הבורר בהודעה. (שם הפרמטר לא משנה לבדוק — `T` או `E`; מה שנבדק הוא האילוץ, שתי פונקציות מיוצאות, ושאות מהן באמת שואלת את העמוד — החתימות שבשלד, עם גוף ריק, אינן עוזר.)

ואז חבר אותו. קובץ JavaScript אחד לפחות מייבא מ-`./dom.js`. ומשתמש בו. עוזר שנכתב ולא נקרא הוא קובץ, לא ארגון מחדש — ויש על זה שורה נפרדת בניקוד.

רשום ב- `PROMPTS.md` שני דברים לפחות שהכלי כתב ושינית, עם הסיבה — בצורה של שורת הסקירה ממחזור 3, ושל הדוגמה `ai-exemplars/exemplars/ai-review-js-good.md` בחומרי הקורס: מה הוא כתב · מה בדקת ואיך · מה שינית ולמה. לפחות אחד מהם צריך להיות אחד מאלה — כי הם יופיעו:

1. `catch (error: any)`. זה התיקון הראשון שהוא מציע כשהוא נתקל ב- `error.name` תחת `strict`, והוא מכבה בדיוק את מה שביקשת. התיקון הנכון הוא `error instanceof Error`.
2. `as` על התשובה. `WikiPayload` as `(await res.json())` — מהדר, קצר, בודק כלום.
3. `@ts-ignore` מעל שדה שלא הוצהר — בדיוק מה שראית במחזור 3 על `this.kind`.
4. `async function getJson<T>(url: string): Promise<T>`. עוטף `fetch` גנרי, נראה מקצועי מאוד, ואין `T` בזמן ריצה. הוא מבטיח טיפוס שהוא לעולם לא בודק, אז כל מי שקורא לו מקבל אובייקט מוקלד לחלוטין שעשוי להיות מסמך שגיאה.

הגרסה הכנה של 4, כי זו לא "אין להשתמש בגנריקה"

```
async function getJson<T>(url: string, guard: (v: unknown) => v is T): Promise<T>;
```

אותן שלוש שורות, פרמטר אחד נוסף, והקורא מספק גם את הטיפוס וגם את ההוכחה. זה ההבדל בין מערכת טיפוסים לבין קישוט.

מדיניות AI לשבוע הזה

שותף למימוש, כמו משבוע 8, ועם אזהרה ספציפית מאוד לשבוע הזה.

העברה ל-`TypeScript` היא הדבר שכלי AI עושה הכי מהר והכי משכנע, והכי קל לא לשים לב שהוא ויתר. הפלט מהדר, הוא צבעוני בעורך, ויש בו מקומות שבהם המהדר כובה בשקט: `any`, `as`, `@ts-ignore`, וגנריקה שמבטיחה טיפוס שהיא לא בודקת. ראית את שלושת הראשונים במחזור 3.

הבקשה שעובדת אומרת מראש מה אסור: "תחת `strict`, בלי `any` בשום מקום, מה שיוצא מ- `res.json()` מוקלד `unknown` ומצומצם בשומר, בלי `as` על ערך שהגיע מהרשת, ובלי `@ts-ignore`".

ואז קרא את הפלט מול המפרט שכתבת, סעיף 5, שורה-שורה — ומחק כל השתקה והרץ שוב. מה שהמהדר מדפיס כשלא משתיקים אותו הוא רשימת העבודה.

חלק א — המפרט — בלי AI. הוא מה שאתה מצפה, לפני שראית תשובה.

בשכבת השותף, מסלול הצ'אט: הדבק את `js/api.js` המלא ואת סעיפים 1-5 של המפרט, בקש קובץ אחד שלם, העתק אותו ל- `ts/api.ts`, הרץ `tsc npx`, ובצע את אותה ביקורת — הציון זהה. אין לך מנוי? הכול עובד גם בצ'אט החינמי, ואין בכך שום חיסרון בקורס.

תיקיה /week-12 במאגר הסמסטר שלך. **הגשה במודל: כתובת המאגר + ה-SHA של הקומיט.**

הקומיטים חשובים כאן במיוחד: המפרט בקומיט אחד, ts/ , בקומיט שאחריו. זו הראיה היחידה שהמפרט נכתב ראשון, והיא שווה נקודות בחלק א.

אחרי ההגשה, העבר את ההעברה הזאת גם ל- /project שלך. **הפרויקט מוגש בסוף שיעור 12**, ודרישות 22–24 שלו הן בדיוק המטלה הזאת — ואיתו AI_USAGE.md , ההצהרה שראית בדוגמה הטובה ובגרועה במחזור 3.

מה מועתק, ומה לא — בפרויקט:

- מעתיק את ts/api.ts ואת ts/dom.ts (והשם Item הופך לישות של הפרויקט שלך).
- **ts/types.ts של הפרויקט נשאר שלך.** הוא מתאר את הישות שלך, ולא את המזוזה — מזג אליו מה ש- ts/api.ts מייבא ממנו (ErrorKind , Suggestion), ואל תדרוס אותו בקובץ של המטלה.
- **js/state.js ו- js/storage.js נשארים JavaScript כתוב ביד — גם בפרויקט המוגש.** אלה הקבצים שהמבחן בשבוע 13 עובד עליהם. **אל תיצור ts/state.ts או ts/storage.ts:** ה- tsconfig.json כותב את הפלט ל- js/ , ו- tsc npx היה מחליף את הקובץ שכתבת בפלט שלו. העברת storage.js ל- ts/storage.ts היתה תרגול של הכיתה בלבד, והיא אינה נדרשת במטלה הזאת וגם לא נכנסת לפרויקט.
- ה- tsconfig.json של הפרויקט (זה שבתיקיית הפתיחה) נשאר, עם שינוי אחד: ts/dom.ts פורש NodeList , ולכן הוסף "DOM.Iterable" ל- lib שלו.

הגשה מאוחרת: הגשה באיחור אינה מותרת ללא אישור מראש בכתב. הגשה באיחור תגרור הפחתה בציון.

סיימת כשכל אלה נכונים

- ☐ .
npx tsc -p tsconfig.json עובר, ו- **strict** עדיין **true** .
- ☐ .
פתחת את js/types.js שנוצר וספרת מה נשאר בו. **רק השומרים.**
- ☐ .
חיפשת any , as ו- ts-ignore בכל ts/ . אפס any , אפס השתקויות, ואף as על התשובה.
- ☐ .
מחקת בכוונה שדה מ- isItem והרצת — ראית שהמהדר **לא** אמר כלום, כי זה זמן ריצה.
- ☐ .
כתבת `const x: string = 5` איפשהו, ראית את השגיאה, ומחקת. **תדע איך היא נראית.**
- ☐ .
כל ייבוא בתוך ts/ מסתיים ב- .js .
- ☐ .
הקלדת מילה בתיבת החיפוש וקיבלת תוצאות. **בדיוק כמו לפני ההעברה.**
- ☐ .
כיבית את ה-Wi-Fi והקלדת. יש משפט, ויש כפתור.
- ☐ .
הקלדת מילה ארוכה מהר. הפאנל **לא האדים** אף פעם.
- ☐ .
git diff על js/api.js מול הקובץ שקיבלת: סוגריים, רווחים ובדיקות typeof שהמהדר ביקש — ואף תנאי שהתוצאה שלו השתנתה.



אף `<script src>` ב-HTML לא השתנה.